

Comparative Performance Analysis of Structurally Similar Virtual Classrooms in Peru Using JMeter and Locust Under Various Load Techniques

Eduardo Maquera Julio Segundo ^{1,*} 

¹ Universidad Nacional del Altiplano, Puno, Peru; jedulao@est.unap.edu.pe (E.M.J.S.)

Abstract

Performance, as a critical quality characteristic, plays a key role in the success of software products. This study presents a comparative performance analysis of fifteen university virtual classrooms in Peru with similar structures, using Apache JMeter and Locust tools under various load techniques. Key metrics such as response time, error rate, requests per second, and resource usage were evaluated to identify strengths, bottlenecks, and opportunities for improvement in each institution's infrastructure. The results show significant differences among the evaluated platforms: universities such as UNE, UNT, and UNSM presented consistent performance with low response times and zero errors, while institutions such as UNAJMA, UNIFSLB, and UNAH showed critical issues regarding slowness and high failure rates. It is concluded that the implementation of load balancers, continuous monitoring, and additional testing are essential measures to improve the reliability and scalability of the evaluated systems.

Keywords: distributed systems; scalability; replication; partitioning; load balancing; load testing; optimization; JMeter; Locust; virtual classrooms

1. Introduction

Performance, as an important quality characteristic, plays a key role in the success of software products. Performance assurance is of great importance, especially in domains where the quality assurance of functional and non-functional aspects of system behavior is essential. For instance, virtual classroom platforms with internet-based user interfaces, such as websites, are examples of systems whose success is subject to performance assurance. Virtual classrooms are usually fundamental parts of universities, and their performance is a prerequisite for the acceptable execution of their functions [1,2].

Improving the performance of distributed systems in current conditions, where there is a need to manage large amounts of data and increase user traffic [3], alongside the expansion of cloud-based applications, the Internet of Things, and artificial intelligence, has become a priority [4,5]. The ability of the system to measure accurately has become a key factor in its operational success and sustainability [6]. Distributed models, which allow collaboration among multiple geographically dispersed units, are a good way to achieve this scale [7,8]. However, as processing and storage requirements increase, new challenges arise that must be managed to maintain efficiency [9].

Scaling design methods studied in the literature, including iteration, data partitioning, and load balancing, have advantages and disadvantages depending on the application context [10,11]. For example, replication improves sensitivity, reduces errors, and can

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the author.

Submitted to *Appl. Sci.* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

decrease the workload by distributing tasks among nodes [12]. However, these methods can also add complexity in terms of data consistency and resource management [13].

Furthermore, this study focuses on evaluating various activities, using load testing and optimization analysis as basic tools to measure the effectiveness of the implemented methods [14]. Load testing allows comparing different conditions and identifying performance bottlenecks, which is important for resource planning and management [3]. By integrating theoretical methods and practical applications, this study considers how design decisions affect the overall performance of distributed systems [4,6].

2. Materials and Methods

In this study, a comparative analysis of various scalable design techniques applied to distributed systems under different workloads was carried out. The methodology was divided into the following key stages:

2.1. Apache JMeter

2.1.1. Load Testing

Load testing tools such as Apache JMeter and Locust.io were used to generate controlled traffic and measure the responsiveness of the systems under different conditions. These tests allowed the identification of bottlenecks and the evaluation of effectiveness in terms of response time and resource usage.

2.1.2. JMeter Configuration

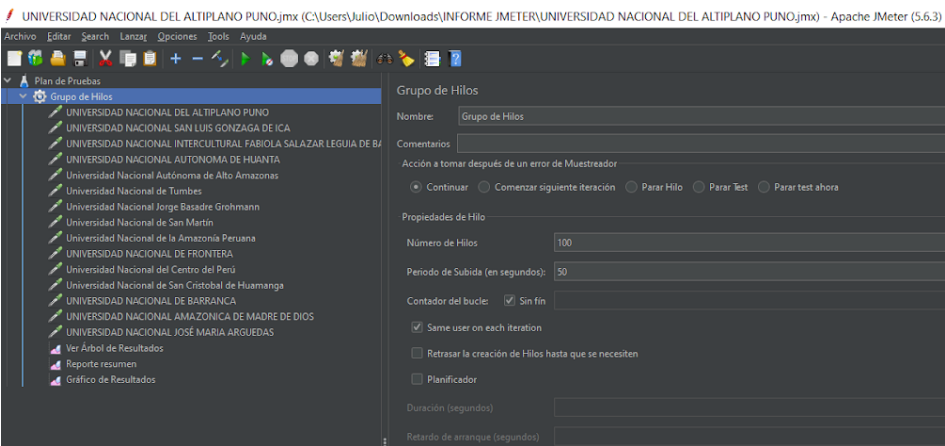


Figure 1. JMeter was used as the main tool to conduct performance tests on educational platforms. The configuration was carefully structured to replicate realistic usage scenarios and evaluate system capacity under different load conditions.

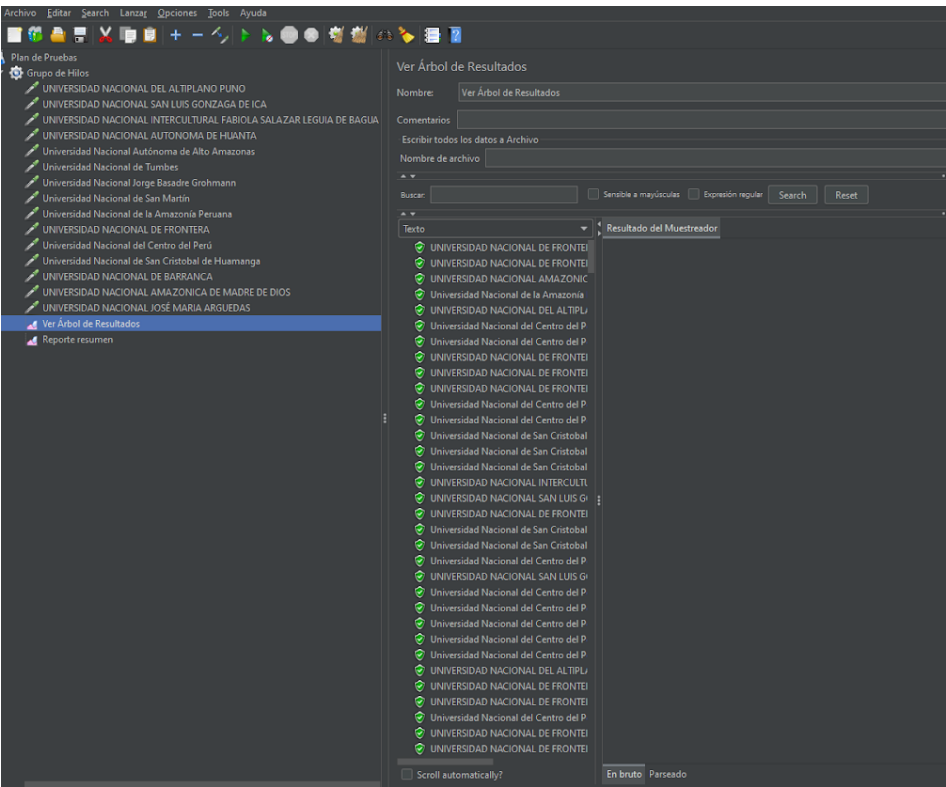


Figure 2. Based on the analysis of the logs and server response results, it is possible to verify whether the requests made to the web application were successful or not. The response code presents the accumulated result from the server since the initial point of contact by the client.

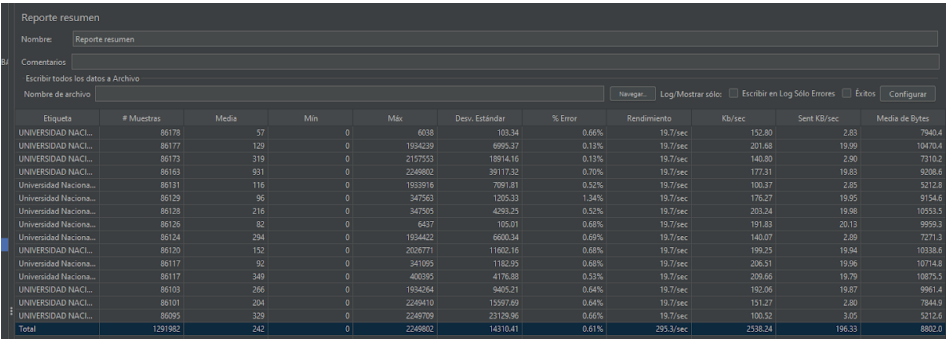


Figure 3. Performance report with statistical data from different universities. It includes a table presenting metrics such as the number of samples, minimum/maximum values, standard deviation, error percentage, throughput, and speed in Kb/sec. The data corresponds to different university URLs and shows an average throughput of 19.77/sec with errors close to 1%.

For the tests conducted with Apache JMeter, the following key elements were configured:

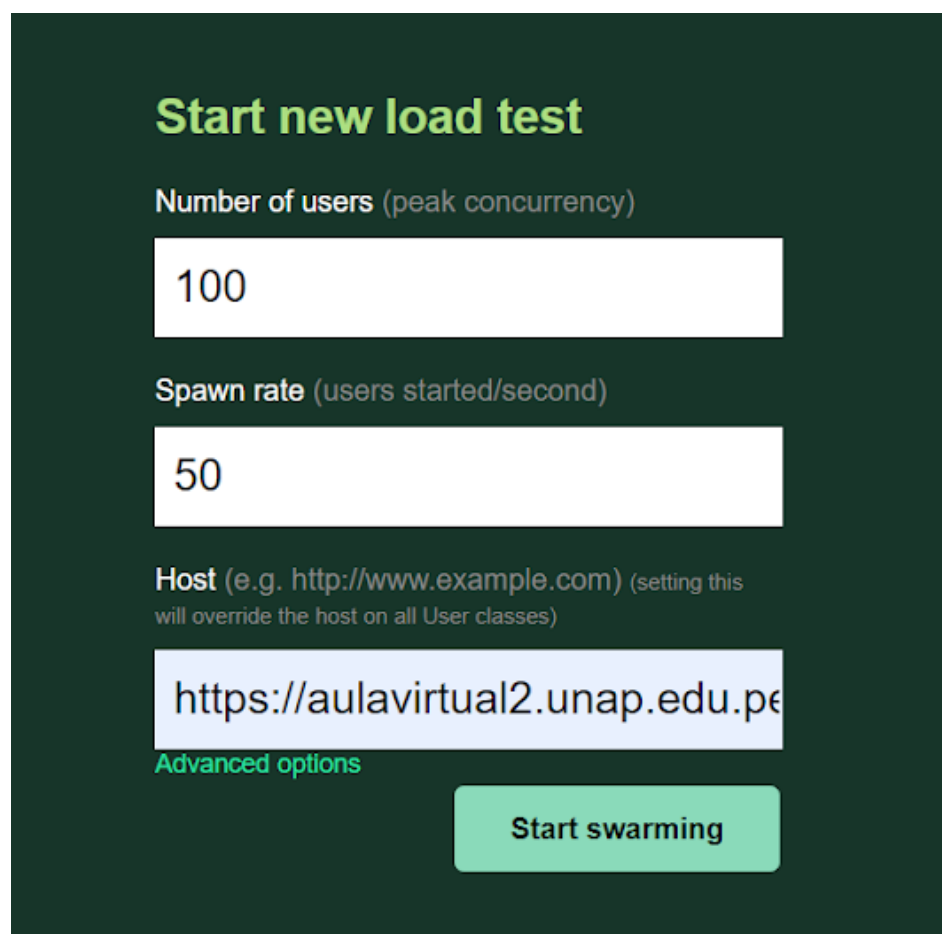
- **Test Plan:** A well-defined test plan organized all activities, including scenarios of concurrent users interacting with the platform through common actions such as logging in, exploring content, and uploading assignments.
- **Threads (users):** Virtual users were simulated by configuring different concurrency levels, from 50 to 100 simultaneous users, allowing the observation of the platform's behavior under different traffic volumes.
- **Test Controllers:** Logic controllers were used to model specific workflows, such as page navigation, content queries, and task processing.

- **Load Scenarios:** To analyze the system's performance in a global context, scenarios simulating access from different geographic locations were included, considering varying latencies and network conditions.
- **Virtual Users:** Users are created virtually, without physical presence, to test the web application and investigate cumulative performance. The **HTTP Request** option is selected because the HTTP protocol allows presenting the result in a web browser or HTTP client.

The results generated by JMeter included detailed data on response times, error rates, and system resource consumption.

2.2. Locust

Locust is a tool developed in Python used to analyze the performance of a web application by simulating access from multiple simultaneous users. This tool is capable of generating traffic that interacts with the web application to evaluate how it responds to high concurrent user loads.



Start new load test

Number of users (peak concurrency)

100

Spawn rate (users started/second)

50

Host (e.g. <http://www.example.com>) (setting this will override the host on all User classes)

[https://aulavirtual2.unap.edu.pe](#)

[Advanced options](#)

Start swarming

Figure 4. The test simulates up to 100 concurrent users interacting with the website, implying that the system must handle 100 simultaneous requests without failure. Additionally, the spawn rate establishes that Locust will start 50 users per second, allowing control over the speed at which virtual users are generated during the load test.



Figure 5. Locust test report; the results reflect the system’s capacity to handle a high load of users and requests, the speed with which it responds, and the volume of data processed under the established test conditions.

2.3. Performance Analysis

Performance was evaluated using key metrics such as:

- Average response time.
- Error rate.
- CPU and memory usage.
- Network latency.

These metrics were collected and analyzed using data visualization tools to highlight the impact of each technique on educational platforms.

2.4. Failure Scenario Simulation

Failures in selected nodes were simulated to evaluate the recovery capacity of the implemented techniques. This included measuring the reconfiguration time and data consistency after failure events.

2.5. Materials Used

- **Hardware:** Virtual servers running in a simulated local network.
- **Software:** Apache JMeter and Locust.io for load testing.

3. Results

3.1. Results with Apache JMeter

The results in Table 1 represent key performance metrics and request behavior in a simulated environment. Below is an interpretation of each column of results.

Table 1. University performance test results using Apache JMeter.

University	# Samples	Mean	Min	Max	Std. Dev.	% Error	Kb/sec	Mean Bytes
UNAP	86,178	57	0	6,038	10,333	6,579	15,280	2,829
UNICA	86,177	129	0	1,934,239	6,995	12,764	2,017	1,999
UNIFSLB	86,173	319	0	2,157,553	18,914	13,229	14,080	2,905
UNAH	86,163	931	0	2,249,802	3,912	6,952	1,773	19,828
UNAAA	86,131	116	0	1,933,916	7,092	521	1,004	2,854
UNT	86,129	96	0	347,563	1,205	1,340	17,627	1,995
UNJBG	86,128	216	0	347,505	4,293	5,248	20,324	19,981
UNSM	86,126	82	0	6,437	10,501	6,804	19,183	20,125
UNAP*	86,124	294	0	1,934,422	6,600	6,874	14,007	28,887
UNF	86,120	152	0	2,026,771	1,160	6,804	19,925	1,994
UNCP	86,117	92	0	341,095	11,829	6,828	2,065	19,965
UNSCH	86,117	349	0	400,395	41,769	5,318	2,097	1,979
UNAB	86,103	266	0	1,934,264	9,405	6,411	19,206	19,865
UNAMAD	86,101	204	0	2,249,410	15,598	642	15,127	2,797
UNAJMA	86,095	329	0	2,249,709	2,313	66	10,052	30,461
Total	1,291,982	242	0	2,249,802	14,310	6,133	25,382	19,633

* Second measurement for UNAP (Universidad Nacional de la Amazonía Peruana).

3.1.1. Interpretation of JMeter Results

- **Label:** Indicates the name of the institution or unit under test. In this case, they are Peruvian universities.
- **# Samples:** Represents the number of requests made (or simulated transactions) during the test. High values indicate that many iterations were performed, useful for analyzing performance patterns.
- **Mean:** This is the average response time in milliseconds for all requests.
 - Low values indicate a fast and efficient system.
 - For example, UNAP has a mean of 57 ms, which is quite fast, while UNAH has a significantly higher mean (931 ms), indicating latency or slower processing.
- **Min and Max:**
 - **Min:** Lowest response time in milliseconds. A value of 0 can indicate immediate responses (likely cached responses).
 - **Max:** Highest recorded time. Extremely high values, such as those of UNICA (1,934,239 ms), could indicate issues like bottlenecks or excessive timeouts.
- **Standard Deviation:** Measures variability in response times. A high value implies an inconsistent experience for users. For example, UNIFSLB shows a very high standard deviation (18,914), reflecting significant performance fluctuations.
- **% Error:** Indicates the percentage of requests that failed during the test. High values are concerning as they reflect reliability issues.
- **Throughput (Kb/sec):** Shows the amount of data processed per second. High throughput indicates efficiency in data transfer. UNJBG, with 20,324 Kb/sec, demonstrates very high throughput compared to other institutions.
- **Mean Bytes:** The average size of responses returned by the server. Larger values typically indicate responses with more data, such as complex web pages or unoptimized responses.

3.1.2. General JMeter Analysis

1. Outstanding institutions:

- **UNAP** and **UNSM** have consistently low average times, indicating good overall performance. 125
 - **UNJBG** stands out for its high data transfer throughput. 126
2. **Identified problems:** 127
- **UNICA** and **UNAH** present high maximum response times, which could indicate bottlenecks or capacity issues. 128
 - Several universities have extremely high standard deviations, such as **UNIFSLB**, reflecting inconsistencies in performance. 129
3. **Recommendations:** 130
- **Performance optimization:** Identify and mitigate bottlenecks in servers of institutions with high maximum and mean times. 131
 - **Error monitoring:** Reduce the error percentage through additional testing and adjustments in exception handling. 132
 - **Consistency:** Reduce standard deviation by implementing infrastructure improvements, such as load balancing or network upgrades. 133

These results provide a basis for prioritizing infrastructure and software improvements, especially for institutions with high response times or marked inconsistencies. 140

3.2. Results with Locust 141

The data obtained in the test with **Locust** (Table 2) reflect the performance and behavior of different university services under load. The fifteen evaluated educational platforms correspond to: 142

- Universidad Nacional del Altiplano Puno (UNAP) 143
- Universidad Nacional San Luis Gonzaga de Ica (UNICA) 144
- Universidad Nacional Intercultural Fabiola Salazar Leguía (UNIFSLB) 145
- Universidad Nacional Autónoma de Huanta (UNAH) 146
- Universidad Nacional Autónoma de Alto Amazonas (UNAAA) 147
- Universidad Nacional de Tumbes (UNT) 148
- Universidad Nacional Jorge Basadre Grohmann (UNJBG) 149
- Universidad Nacional de San Martín (UNSM) 150
- Universidad Nacional de la Amazonía Peruana (UNAP) 151
- Universidad Nacional de Frontera (UNF) 152
- Universidad Nacional del Centro del Perú (UNCP) 153
- Universidad Nacional de San Cristóbal de Huamanga (UNSCH) 154
- Universidad Nacional de Barranca (UNAB) 155
- Universidad Nacional Amazónica de Madre de Dios (UNAMAD) 156
- Universidad Nacional José María Arguedas (UNAJMA) 157

Table 2. University performance test results using Locust.

University	# Requests	# Fails	Average (ms)	Maximum (ms)	RPS
UNAP	23,917	98	7,382	2,259,493	8.5
UNICA	24,681	0	146	2,776	40.8
UNIFSLB	5,158	5,158	538	10,171	18.9
UNAH	10,504	10,504	957	4,891	16.3
UNAAA	15,392	46	352	63,908	41.7
UNT	13,431	0	106	4,219	46.4
UNJBG	20,503	0	71	2,376	47.5
UNSM	23,586	0	83	1,814	47.3
UNAP*	16,522	16,522	1,454	7,134	27.4
UNF	42,925	0	60	2,468	48.1
UNCP	29,982	0	87	4,359	47.3
UNSCH	13,321	0	135	1,832	46.1
UNAB	33,206	33,206	1,306	7,457	28.7
UNAMAD	16,370	0	112	2,562	46.2
UNAJMA	56,901	200	11,271	6,789,815	6.2

* Second measurement for UNAP (Universidad Nacional de la Amazonía Peruana).

3.2.1. Interpretation of Locust Test Results

- Number of Requests (# Requests):** Indicates how many requests were made to the system during the test. Universities with a high number of requests, such as **UNAJMA** (56,901 requests), show that their service was highly demanded during the testing period.
- Number of Errors (# Fails):** A high number of failures suggests problems in load handling or errors in services. **UNIFSLB** had 5,158 failures, indicating that all requests failed, possibly due to an unavailable or misconfigured system. Conversely, universities like **UNT** presented no errors, reflecting high reliability.
- Average Response (Average ms):** Measures the average time in milliseconds the system took to respond to a request. **UNAJMA** has an extremely high average (11,271 ms), indicating severe slowness, likely due to overload or an inefficient system. Universities like **UNF** have much lower average times (60 ms), reflecting an agile system.
- Maximum Response Time (Maximum ms):** Indicates the longest time the system took to respond to a request. **UNAJMA** recorded a disproportionately high maximum time (6,789,815 ms), signaling a critical failure during the test. In contrast, other universities like **UNJBG** had a much more manageable maximum time (2,376 ms).
- Requests per Second (RPS):** Measures how many requests per second the system can handle. Universities like **UNF** reached high values (48.1 RPS), reflecting the capacity to handle more simultaneous load. Universities with low values, like **UNAP** (8.5 RPS), might need optimization in their infrastructure.

4. Discussion

The results obtained from the tests conducted with JMeter highlight both strengths and weaknesses in the evaluated systems of Peruvian universities. The collected information provides a comprehensive overview of the performance and reliability of each institution, allowing the identification of key patterns:

- Outstanding Performance:**
 - Universities like **UNAP** and **UNSM** show stable performance and consistently low average response times. These results indicate proper infrastructure design and optimization in request handling.

- Likewise, **UNJBG** stands out for its data transfer capacity, reflecting efficiency in processing large volumes of information.
2. **Problematic Areas:**
 - Institutions like **UNICA** and **UNAH** present extremely high maximum response times, suggesting severe bottlenecks or capacity limitations in their systems.
 - **UNIFSLB** stands out negatively due to its high standard deviation, denoting unstable behavior and significant fluctuations in response time.
 3. **Errors in Tests:**
 - Several universities showed concerning error percentages, pointing to failures in server capacity to handle requests effectively.
 4. **Consistency and Scalability:**
 - Variability in response times and performance among institutions highlights the need to establish better practices in infrastructure design.
 - Universities with good results, such as **UNT** and **UNF**, evidence that implementing scalable and efficient solutions can translate into substantial improvements.
- General conclusions on Locust tests:
- **Stability and Reliability:** Universities like **UNT** and **UNF** show optimal performance with zero errors and low response times.
 - **Areas for Improvement:** Universities like **UNIFSLB** and **UNAH** have a high number of failures, indicating critical issues that must be addressed before considering their systems reliable for real use.
 - **Slowness Issues:** Institutions like **UNAJMA** present extremely high response times and need significant revisions to improve performance.
 - **Outstanding Performance:** Systems like **UNF**'s handled more than 48 requests per second with minimal response times, indicating an efficient infrastructure.
- Technical recommendations:
- **Backend Optimization:** Improve code efficiency and database queries.
 - **Scalability:** Implement load balancers and scale horizontally where necessary.
 - **Active Monitoring:** Use monitoring tools to identify bottlenecks in real-time.
 - **Additional Testing:** Execute more tests with different configurations and loads to confirm results.
- ## 5. Conclusions
- Based on the results obtained, the following conclusions can be drawn:
1. **Identified Strengths:** Universities like **UNAP**, **UNSM**, and **UNJBG** demonstrate that it is possible to achieve solid and consistent performance through optimized and well-designed infrastructures.
 2. **Opportunities for Improvement:** Institutions with high response times and high variability, such as **UNICA**, **UNAH**, and **UNIFSLB**, should focus their efforts on optimizing their servers, identifying bottlenecks, and adopting measures that improve system stability.
 3. **Technical Recommendations:**
 - It is recommended to implement load balancers, improve exception handling in applications, and conduct additional testing to ensure system reliability and consistency.
 - Furthermore, establishing continuous monitoring is crucial to identify problems in real-time and take corrective actions.

4. **Impact on the User:** Institutions that presented critical issues must prioritize the identified improvements, as system performance and reliability are key factors in ensuring a satisfactory user experience and avoiding service interruptions.

Author Contributions: The author has read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data supporting the reported results are available upon request from the corresponding authors.

Acknowledgments: The authors thank the Universidad Nacional del Altiplano Puno for the institutional support provided for the development of this study.

Conflicts of Interest: The authors declare no conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

JMeter	Apache JMeter (performance testing tool)
RPS	Requests Per Second
UNAP	Universidad Nacional del Altiplano Puno / Universidad Nacional de la Amazonía Peruana
UNICA	Universidad Nacional San Luis Gonzaga de Ica
UNIFSLB	Universidad Nacional Intercultural Fabiola Salazar Leguía
UNAH	Universidad Nacional Autónoma de Huanta
UNAAA	Universidad Nacional Autónoma de Alto Amazonas
UNT	Universidad Nacional de Tumbes
UNJBG	Universidad Nacional Jorge Basadre Grohmann
UNSM	Universidad Nacional de San Martín
UNF	Universidad Nacional de Frontera
UNCP	Universidad Nacional del Centro del Perú
UNSCH	Universidad Nacional de San Cristóbal de Huamanga
UNAB	Universidad Nacional de Barranca
UNAMAD	Universidad Nacional Amazónica de Madre de Dios
UNAJMA	Universidad Nacional José María Arguedas

References

1. Jain, S.; Saxena, A.K. A survey of load balancing challenges in cloud environment. In Proceedings of the 2016 International Conference System Modeling & Advancement in Research Trends (SMART), Jan. 2016. <https://doi.org/10.1109/sysmart.2016.7894537>.
2. Jogalekar, P.P.; Woodside, C.M. A scalability metric for distributed computing applications in telecommunications. In *Teletraffic science and engineering*; 1997; pp. 101–110. [https://doi.org/10.1016/s1388-3437\(97\)80016-9](https://doi.org/10.1016/s1388-3437(97)80016-9).
3. Maheshwari, S.; Raychaudhuri, D.; Seskar, I.; Bronzino, F. Scalability and Performance Evaluation of Edge Cloud Systems for Latency Constrained Applications. In Proceedings of the 2018 IEEE/ACM Symposium on Edge Computing (SEC), Oct. 2018, pp. 286–299. <https://doi.org/10.1109/sec.2018.00028>.
4. Dave, A.; Patel, B.; Bhatt, G. Load balancing in cloud computing using optimization techniques: A study. In Proceedings of the 2016 International Conference on Communication and Electronics Systems (ICCES), Oct. 2016, pp. 1–6. <https://doi.org/10.1109/cesys.2016.7889883>.
5. Gupta, S.; Singh, N. Resource Management with Load Balancing Strategies in Fog-IoT Computing Environment: Trends, Challenges and Future directions. In Proceedings of the 2023 International Conference on Artificial Intelligence and Smart Communication (AISC), Jan. 2023, pp. 1358–1359. <https://doi.org/10.1109/aisc56616.2023.10085287>.
6. Gures, E.; Shayea, I.; Ergen, M.; Azmi, M.H.; El-Saleh, A.A. Machine Learning-Based Load Balancing Algorithms in Future Heterogeneous Networks: A Survey. *IEEE Access* **2022**, *10*, 37689–37717. <https://doi.org/10.1109/access.2022.3161511>.

7. Hogade, N.; Pasricha, S. A Survey on Machine Learning for Geo-Distributed Cloud Data Center Management. *IEEE Transactions on Sustainable Computing* **2022**, *8*, 15–31. <https://doi.org/10.1109/tsusc.2022.3208781>. 271
8. Moghadam, M.H.; et al.. Performance Testing Using a Smart Reinforcement Learning-Driven Test Agent. In Proceedings of the 2022 IEEE Congress on Evolutionary Computation (CEC), Jun. 2021, Vol. 518, pp. 2385–2394. <https://doi.org/10.1109/cec45853.2021.9504763>. 272
9. Pradeep, S.; Sharma, Y.K. A Pragmatic Evaluation of Stress and Performance Testing Technologies for Web Based Applications. In Proceedings of the 2019 Amity International Conference on Artificial Intelligence (AICAI), Feb. 2019, pp. 399–403. <https://doi.org/10.1109/aicai.2019.8701327>. 273
10. Wang, J.; Wu, J. Research on Performance Automation Testing Technology Based on JMeter. In Proceedings of the 2019 International Conference on Robots & Intelligent System (ICRIS), Jun. 2019, pp. 55–58. <https://doi.org/10.1109/icris.2019.00023>. 274
11. Abbas, R.; Sultan, Z.; Bhatti, S.N. Comparative analysis of automated load testing tools: Apache JMeter, Microsoft Visual Studio (TFS), LoadRunner, Siege. In Proceedings of the 2017 International Conference on Communication Technologies (ComTech), Apr. 2017, pp. 39–44. <https://doi.org/10.1109/comtech.2017.8065747>. 275
12. Lenka, R.K.; Mamgain, S.; Kumar, S.; Barik, R.K. Performance Analysis of Automated Testing Tools: JMeter and TestComplete. In Proceedings of the 2018 International Conference on Advances in Computing, Communication Control and Networking (ICACCCN), Oct. 2018, pp. 399–407. <https://doi.org/10.1109/icacccn.2018.8748521>. 276
13. Radhakrishna, S.; Nachamai, M. Performance inquisition of web services using soap UI and JMeter. In Proceedings of the 2017 IEEE International Conference on Current Trends in Advanced Computing (ICCTAC), Mar. 2017. <https://doi.org/10.1109/icctac.2017.8249993>. 277
14. Tiwari, V.; Upadhyay, S.; Goswami, J.K.; Agrawal, S. Analytical Evaluation of Web Performance Testing Tools: Apache JMeter and SoapUI. In Proceedings of the 2022 IEEE 11th International Conference on Communication Systems and Network Technologies (CSNT), Apr. 2023, pp. 519–523. <https://doi.org/10.1109/csnt57126.2023.10134699>. 278

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content. 279